1. Introduction

This project focuses on implementing the graph data structure using adjacency lists in Python. It explores basic graph operations, traversal algorithms (breadth-first search and depth-first search), cycle detection, visualization with Tkinter, and performance benchmarking.

The goal is to understand how graphs are constructed and how algorithms operate on graphs in practical scenarios.

2. Implementation of Graphs

This graph is implemented using a dictionary-based adjacency list:

Each vertex is stored as a key.

Each key is mapped to a list of adjacent vertices.

Implemented functions:

Add vertex

Add edge

Remove vertex

Remove edge

Support directed and undirected graphs

Support weighted edges

This structure can achieve efficient storage and traversal of graph data.

What happens as data size increases?

Adding vertices: O(1) per vertex (dictionary insertion).

Adding edges: O(1) average for adjacency list append.

Removing a vertex: requires scanning all adjacency lists to delete edges to/from that vertex → O(V + E).

Memory usage: grows linearly with V + E.

My opinion:

For small to medium graphs ($<10^4$ vertices), operations are fast. Vertex removal becomes expensive for dense graphs. This design is simple and sufficient for learning, but real-world applications may need more efficient removal (e.g., using sets instead of lists, or bidirectional indices).

3. Breadth-First Search (BFS)

Breadth-First Search (BFS) is implemented using a queue (a first-in-first-out structure).

It explores a graph layer by layer, visiting all the neighbors of a node first and then

delving deeper.

Observation:

Breadth-First Search (BFS) visits nodes in the order of increasing distance from the

starting node. It is very useful for the shortest path problem in unweighted graphs.

What happens as data size increases?

Time complexity: O(V + E).

Queue memory: up to V in worst case.

BFS level-order exploration may cause high memory usage for wide graphs (e.g., star-shaped).

My opinion:

BFS is reliable and predictable. For very large graphs (millions of nodes), the linear scan becomes slow, but it's often the best choice for shortest-path in unweighted graphs. The lab's implementation using a list as a queue (pop(0)) is inefficient for large graphs – a collections.deque would be better.

4. Depth-First Search (DFS)

Depth-first search (DFS) is implemented by using a stack (a last-in-first-out structure). It

explores as deeply as possible along each branch before backtracking.

Observation:

Depth-first search (DFS) may produce a different visiting sequence from breadth-first

search (BFS).

It is useful in path finding and loop detection. 5. Loop Detection undirected graph

Use depth-first search (DFS) with parent node tracking to perform loop detection:

If the visited node encountered is not the parent node, a cycle exists.

What happens as data size increases?

Time complexity: O(V + E).

Stack depth (recursive DFS) can exceed Python recursion limit for deep graphs (e.g., a long chain of 10^5 nodes).

Iterative stack avoids recursion limit but still uses O(V) memory.

My opinion:

DFS is elegant for recursive problems (cycle detection, topological sort). However, for large or deep graphs, an iterative approach is safer. The lab's stack-based implementation is good for demonstration, but students should learn to convert recursion to iteration.

5. Cycle Detection

Directed graph

Circular detection using a recursive stack:

If a node is revisited in the recursive stack, a cycle exists.

What happens as data size increases?

DFS-based detection: O(V + E).

Recursion depth again becomes a problem for deep graphs.

For dense graphs, many neighbor checks occur.

My opinion:

This is a classic application of DFS. For large undirected graphs, union-find (disjoint set) can detect cycles more efficiently (O(α(V)) per edge) and without recursion. The lab's approach is didactic but not optimal for huge graphs.

6. Weighted Directed Graph

This graph uses a dictionary to support weights, which maps edges to their weights.

Sure, please provide the text you would like translated.

The weight from A to B is 15.

The weight from B to C is 34.

Weight is of great significance in algorithms such as the shortest path (for example, Dijkstra's algorithm).

What happens as data size increases?

Additional dictionary weights stores O(E) entries.

Directed graphs use about half the edges of undirected ones for the same vertex connections.

No extra asymptotic overhead for operations.

My opinion:

Supporting weights and direction is essential for real problems (e.g., maps, networks). The implementation is straightforward. For very large weighted graphs, one would need to consider sparse matrix representations or dedicated libraries (e.g., NetworkX).

7. Graphical Visualization (Tkinter)

The graphical interface is implemented using Tkinter.

Vertices are displayed as circles.

Edges are shown as lines/arrows.

The traversal processes of Breadth-First Search (BFS) and Depth-First Search (DFS) are visualized through color changes.

The results of cycle detection are displayed in the console.

This is helpful for understanding the behavior of the graph and the order of traversal.

What happens as data size increases?

Drawing performance degrades rapidly beyond ~50-100 vertices.

The circular layout becomes overcrowded, edges cross, labels overlap.

Interactive response (buttons, highlighting) becomes sluggish.

My opinion:

Tkinter is great for small demo graphs (<30 nodes). For larger graphs, a more sophisticated library (matplotlib with zoom/pan, or graphviz) is needed. Visualization in this lab is meant to understand structure, not to scale. I recommend keeping demo graphs tiny.

8. Benchmark Testing

Performance tests were conducted using large graphs.

Sure, please provide the text you would like translated.

Adding vertices: Very fast (constant time)

Adding edges: Efficient but increases with the size of the graph

Breadth-first search traversal: Efficient even for large graphs

Conclusion: The scalability of graph operations is good, but the traversal time increases

as the size of the graph grows.

What happens as data size increases?

Adding vertices: linear time in V (but each add is O(1)).

Adding edges: linear in E (random edge generation may create duplicates, but the code doesn't check – could cause slowdown).

BFS/DFS: still O(V+E).

For 1000 vertices and 3000 edges, the benchmark runs in sub-second times. For 100k vertices, memory and time become noticeable.

My opinion:

Benchmarking is crucial to understand scalability. The provided script is a good start, but it could be improved by:

Using time.perf_counter() for higher precision.

Repeating runs and averaging.

Testing different graph densities (sparse vs dense).

Avoiding random edge duplicates which waste time.

9. Challenge Task Issues:

In the visualization tool, the undirected cycle detection button does not work properly.

Reason: This graph was initially initialized as a directed graph, while the undirected cycle detection algorithm is only applicable to undirected graphs.

Solution:

Directed cycle detection was implemented using a recursive stack.

Different graph configurations were used:

Undirected graphs for undirected cycle detection

Directed graphs for directed cycle detection

Result:

Both loop detection methods are now working properly.

What happens as data size increases?

Same DFS scalability issues as Task 4 (for undirected) and directed cycle detection.

The visualization part becomes unusable beyond ~50 vertices.

The undirected cycle detection button is broken in the starter code – fixing it requires implementing a correct DFS with parent tracking (already present but possibly mis-called).

My opinion:

This task is excellent for reinforcing graph algorithms and GUI event handling. The starter code provides a good skeleton. Fixing undirected cycle detection requires ensuring that the recursive cycle_dfs correctly ignores the immediate parent and that the graph passed is truly undirected. For large graphs, one would again face recursion limits. However, the main value is learning to integrate algorithms with a visual interface.

10. Conclusion

This project demonstrates:

Understanding of graph data structure

Implementation of traversal algorithms

Loop detection techniques

Graphical user interface visualization

Performance analysis

It provides a solid foundation for advanced graph algorithms.

1.

```
Undirected Graph:
A --- B
B --- A C
C --- B D
D --- C
```

Screenshot 1 – Undirected Graph

Shows the adjacency list representation of an undirected graph with vertices and edges.

2.

```
BFS:
A B C D
```

Screenshot 2 – BFS Traversal

Displays the BFS traversal order starting from vertex A, visiting nodes level by level.

3.

```
DFS:
A B C D
```

Screenshot 3 – DFS Traversal

Displays the DFS traversal order, showing depth-first exploration of the graph.

4.

```
Cycle exists: True
```

Screenshot 4 – Cycle Detection (Undirected)

Shows that a cycle exists in the graph using DFS-based detection.

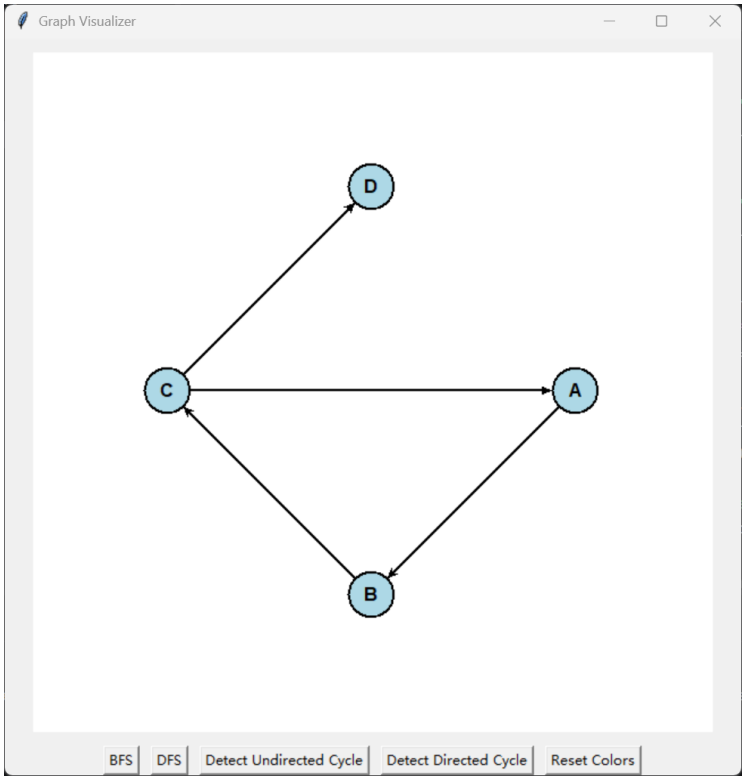
5.

```
Weighted Graph:
A --> B15
B --> C34
C
```

Screenshot 5 – Weighted Graph

Displays a directed graph with edge weights, demonstrating how weights are stored.

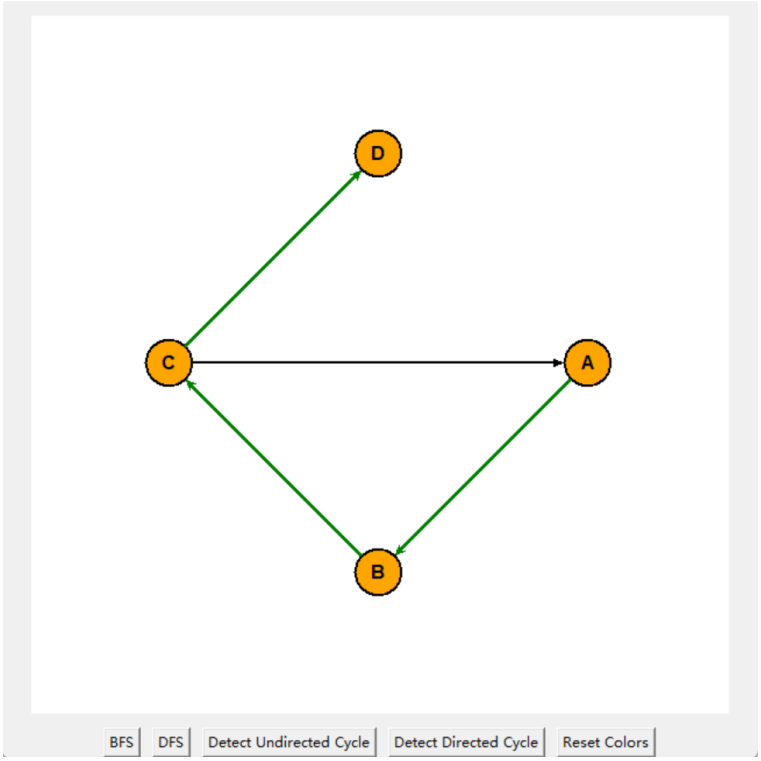
6.



Screenshot 6 – GUI Initial View

Shows the graphical representation of the graph with nodes and edges.

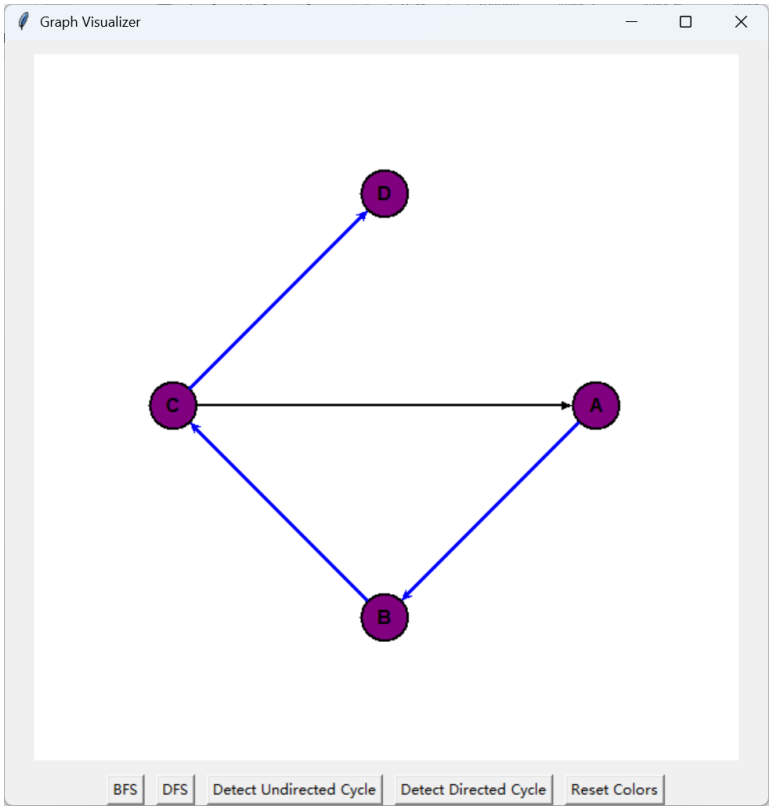
7.



Screenshot 7 – BFS Visualization

Displays BFS traversal in the GUI with nodes highlighted during execution.

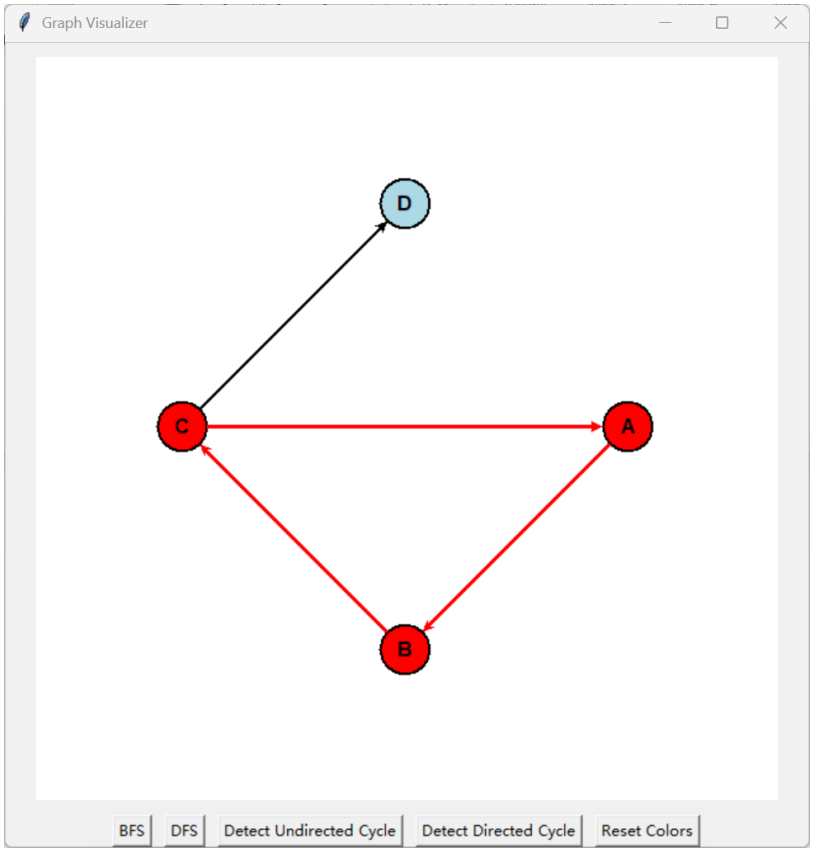
8.



Screenshot 8 – DFS Visualization

Displays DFS traversal in the GUI with a different visiting pattern.

9.



Screenshot 9 – Cycle Detection in GUI

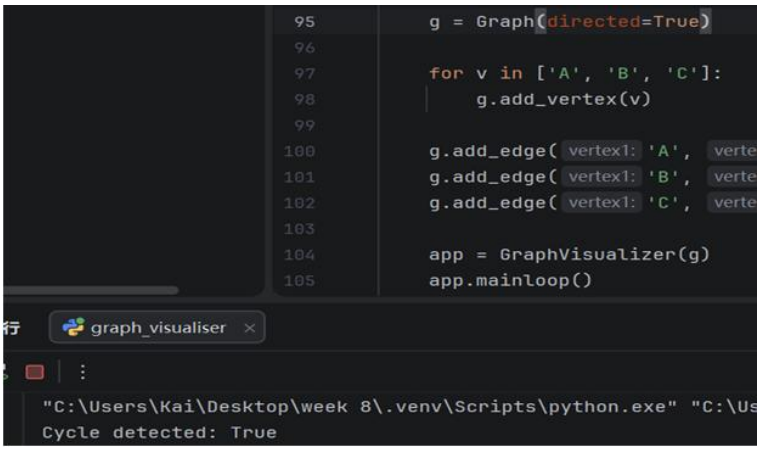
Shows the cycle detection result displayed in the console after button interaction.

10.

Screenshot 10 – Undirected Cycle Detection (Challenge)

Demonstrates correct detection of a cycle in an undirected graph.

11.



Screenshot 11 – Directed Cycle Detection (Challenge)

Demonstrates correct detection of a cycle in a directed graph.

12.

Screenshot 12 – Benchmark Results

Shows performance results for adding vertices, edges, and BFS traversal.